

Taller 3 – Programación discreta

Criptografía, grafos, álgebra de Boole, Shannon y computación cuántica

Jean Carlo Triana Guzmán

jetrianag@unal.edu.co

Diego Alejandro Rodríguez Sandoval

dirodriguezsa@unal.edu.co

Matemáticas Discretas I, Universidad Nacional de Colombia

Resumen—Este documento presenta la solución al Taller 3 del curso Matemáticas Discretas I. Se implementaron diez programas cortos que muestran cómo distintas ideas del curso –aritmética modular, teoría de grafos, álgebra de Boole, teoría de la información y álgebra lineal básica– se convierten en código funcional y verificable. Para cada punto se explica el problema que resuelve, la idea matemática detrás de la solución, cómo se ejecuta, qué pruebas se hicieron y qué limitaciones tiene.

I. Introducción

Las matemáticas discretas no son solo definiciones para memorizar: son la base de herramientas que se usan todos los días en seguridad informática, redes, electrónica digital y computación. Este taller busca cerrar esa brecha entre la teoría vista en clase y la práctica, programando diez problemas pequeños que ilustran esas conexiones.

El repositorio se organiza en tres bloques: **Bloque A** (criptografía: cifrado César, RSA de juguete y un protocolo básico de cómputo multipartita seguro), **Bloque B** (grafos: ruta más corta, impacto del cierre de un nodo y coloreo de grafos) y **Bloque C** (álgebra de Boole, entropía de Shannon y un primer simulador cuántico de un qubit). Todo el código está escrito en Python 3, usando exclusivamente la librería estándar (`math`, `random`, `heapq`, `itertools`), y cada programa cuenta con pruebas automatizadas ubicadas en la carpeta `tests/` del repositorio.

II. Cifrado César

II-A. ¿Qué problema resuelve el programa?

Resuelve el problema de ocultar un mensaje de texto mediante una transformación simple y reversible, y también el problema inverso: recuperar un mensaje cifrado con este método cuando no se conoce la llave.

II-B. ¿Qué idea matemática usa?

El cifrado César es una operación de **aritmética modular** sobre el alfabeto. Cada letra se identifica con un número entre 0 y 25 (su posición en el alfabeto) y cifrar

consiste en sumar un desplazamiento k módulo 26:

$$C_i \equiv (M_i + k) \pmod{26}, \quad M_i \equiv (C_i - k) \pmod{26}.$$

Descifrar es exactamente la operación inversa en el grupo $(\mathbb{Z}_{26}, +)$: sumar $-k$ módulo 26 deshace el desplazamiento original. El programa implementa esto en `desplazar_caracter`, que calcula la posición del carácter dentro de su bloque (mayúsculas o minúsculas) y aplica el módulo.

II-C. ¿Cómo se ejecuta?

```
python3 src/cripto/cesar.py
```

El programa despliega un menú con cuatro opciones: cifrar un texto dado k , descifrarlo, probar los 26 desplazamientos posibles (fuerza bruta) y salir.

II-D. ¿Qué pruebas hicieron?

Las pruebas (`tests/cesar_test.py`) verifican: (1) el caso mínimo del enunciado, HOLA UNAL con $k = 3$ produce KROD XQDO; (2) que descifrar el resultado de cifrar con la misma k devuelve el texto original; (3) que los espacios, signos de puntuación y números se conservan sin alterar; (4) que mayúsculas y minúsculas se desplazan de forma independiente ("aA" con $k = 1$ da "bB"); y (5) que la fuerza bruta siempre incluye, entre sus 26 resultados, el texto original.

II-E. ¿Qué limitaciones tiene la solución?

Solo cifra letras del alfabeto latino sin ñ; cualquier otro carácter (tildes, ñ, símbolos) pasa sin modificar. Al ser un cifrado monoalfabético con solo 26 llaves posibles, es trivialmente vulnerable a fuerza bruta y también a análisis de frecuencias, por lo que no ofrece ninguna seguridad real; su valor aquí es puramente didáctico.

III. RSA de juguete

III-A. ¿Qué problema resuelve el programa?

Muestra, con números pequeños, cómo un sistema de cifrado asimétrico permite cifrar un mensaje con una llave pública y descifrarlo únicamente con la llave privada correspondiente, sin necesidad de compartir un secreto previo.

III-B. ¿Qué idea matemática usa?

RSA se basa en **aritmética modular** y en el teorema de Euler-Fermat. Dados dos primos p, q , se calcula $n = pq$ y $\varphi(n) = (p-1)(q-1)$. Se elige un exponente público e tal que $\gcd(e, \varphi(n)) = 1$, y su inverso modular d (tal que $ed \equiv 1 \pmod{\varphi(n)}$) se obtiene con el **algoritmo de Euclides extendido**, implementado recursivamente en `euclides_extendido`. Cifrar y descifrar son exponenciaciones modulares:

$$C \equiv M^e \pmod{n}, \quad M \equiv C^d \pmod{n}.$$

Que esto funcione depende de que d sea efectivamente el inverso de e módulo $\varphi(n)$, lo cual solo existe si e y $\varphi(n)$ son primos relativos; por eso el programa valida $\gcd(e, \varphi(n)) = 1$ antes de aceptar la llave.

III-C. ¿Cómo se ejecuta?

```
1 python3 src/cripto/rsa.py
```

El menú permite generar llaves a partir de p, q y e , cifrar un mensaje M y descifrar un cifrado C con las llaves activas.

III-D. ¿Qué pruebas hicieron?

El caso obligatorio del enunciado ($p = 61, q = 53, e = 17, M = 65$) se verifica exactamente: $n = 3233, \varphi(n) = 3120, d = 2753, C = 2790$ y el descifrado recupera $M = 65$. También se prueba que el algoritmo de Euclides extendido satisface la identidad de Bézout ($ax + by = \gcd(a, b)$) para $a = 240, b = 46$, y que un exponente e que comparte factor con $\varphi(n)$ (por ejemplo $e = 6$) es rechazado con un `ValueError`.

III-E. ¿Qué limitaciones tiene la solución?

Usa primos pequeños elegidos a mano, sin ninguna prueba de primalidad ni generación aleatoria segura; con números de este tamaño n se puede factorizar en milisegundos, así que esta implementación no debe usarse como seguridad real. Tampoco se implementa relleno (*padding*) como OAEP, que en un RSA real es indispensable para evitar ataques.

IV. MPC básico: suma secreta

IV-A. ¿Qué problema resuelve el programa?

Permite calcular la suma y el promedio de una lista de notas entre varios servidores, sin que ninguno de ellos vea en ningún momento las notas originales de cada estudiante.

IV-B. ¿Qué idea matemática usa?

Se basa en **aritmética modular** y en compartir un secreto mediante partes aleatorias (*secret sharing* aditivo). Cada nota x se reparte en tres partes s_1, s_2, s_3 elegidas de forma que

$$x \equiv s_1 + s_2 + s_3 \pmod{M},$$

generando s_1 y s_2 al azar en $[0, M)$ y despejando $s_3 \equiv (x - s_1 - s_2) \pmod{M}$. Cada servidor recibe una sola parte de cada nota (nunca las tres), por lo que individualmente no puede reconstruir x : una única parte es un número aleatorio que no revela nada por sí solo. Al final, cada servidor suma sus partes módulo M , y sumando esas tres sumas parciales módulo M se reconstruye la suma total real, gracias a que la suma módulo M es lineal.

IV-C. ¿Cómo se ejecuta?

```
python3 src/cripto/mpc_basico.py
```

Se ingresa una lista de notas separadas por coma y el programa muestra las partes que recibiría cada uno de los tres servidores, sus sumas parciales, y finalmente la suma total y el promedio reconstruidos.

IV-D. ¿Qué pruebas hicieron?

Se verifica que las tres partes generadas para una nota siempre suman esa nota módulo M ; que el ejemplo mínimo del enunciado (notas $[40, 35, 50, 25]$) produce suma 150 y promedio 37.5; que, para una nota dada, al menos una de sus tres partes es distinta de la nota original (es decir, ningún servidor individual "ve" el valor real); y que pedir el promedio de una lista vacía lanza un error controlado.

IV-E. ¿Qué limitaciones tiene la solución?

Es una simulación educativa, no un protocolo criptográfico auditado: los tres "servidores" corren en el mismo proceso, por lo que no hay comunicación real ni garantías contra colusión entre servidores (si dos de los tres se ponen de acuerdo, sí podrían reconstruir la nota).

Tampoco se garantiza que el reparto de partes sea criptográficamente aleatorio (se usa `random`, no un generador seguro).

V. Ruta más corta (Dijkstra)

V-A. ¿Qué problema resuelve el programa?

Encuentra la distancia mínima y la ruta más corta entre dos puntos de una red de transporte representada como un grafo ponderado.

V-B. ¿Qué idea matemática usa?

Usa **teoría de grafos**: la ciudad se modela como un grafo no dirigido $G = (V, E)$ donde los vértices son lugares y las aristas tienen un peso (tiempo o distancia). El **algoritmo de Dijkstra** mantiene una distancia tentativa para cada vértice (inicialmente ∞ , salvo el origen que es 0) y, usando una cola de prioridad (`heapq`), va extrayendo siempre el vértice no visitado con menor distancia tentativa y *relajando* sus aristas: si llegar a un vecino a través del vértice actual es más corto que lo que se tenía registrado, se actualiza esa distancia. El algoritmo termina con las distancias mínimas reales desde el origen a todos los vértices alcanzables.

V-C. ¿Cómo se ejecuta?

```
1 cd src/grafos
2 python3 dijkstra.py
```

El script arma un grafo de prueba con 8 vértices y 12 aristas (Portal, Calle26, Museo, Centro, Universidad, Parque, Estadio, Terminal) y calcula la ruta más corta entre Museo y Universidad.

V-D. ¿Qué pruebas hicieron?

Se verifica que la distancia de Portal a Parque es 32, con ruta Portal $\rightarrow$ Museo $\rightarrow$ Estadio $\rightarrow$ Terminal $\rightarrow$ Parque; que la distancia de Museo a Universidad es 23, con ruta Museo $\rightarrow$ Calle26 $\rightarrow$ Centro $\rightarrow$ Universidad; y que la distancia de un vértice a sí mismo es 0.

V-E. ¿Qué limitaciones tiene la solución?

Dijkstra requiere que todos los pesos sean no negativos: si existiera una arista con peso negativo, la suposición de que el primer vértice extraído de la cola ya tiene su distancia mínima definitiva dejaría de ser válida, y el algoritmo podría dar resultados incorrectos. Un camino "óptimo" aquí significa mínima suma de pesos, no necesariamente el camino con menos paradas. Además la

implementación no soporta grafos dirigidos ni pesos variables en el tiempo.

VI. Cierre de una estación

VI-A. ¿Qué problema resuelve el programa?

Mide el impacto de cerrar un punto de la red (por ejemplo, una estación) sobre las rutas más cortas entre varios pares de lugares: cuáles rutas se alargan y cuáles quedan completamente desconectadas.

VI-B. ¿Qué idea matemática usa?

Reutiliza el mismo modelo de grafo ponderado y el algoritmo de Dijkstra del punto anterior. Cerrar una estación corresponde a **eliminar un vértice** (y todas sus aristas incidentes) del grafo, obteniendo un subgrafo $G' = G - v$. Comparar las distancias más cortas en G y en G' para los mismos pares origen-destino permite cuantificar el impacto: si la distancia en G' es mayor que en G , la ruta se alargó; si en G' ya no existe camino (distancia ∞), el par quedó desconectado.

VI-C. ¿Cómo se ejecuta?

```
1 cd src/grafos
2 python3 cerrar_estacion.py
```

Usa el mismo grafo de 8 vértices y 12 aristas del punto anterior, cierra el vértice Centro y compara cinco pares origen-destino antes y después del cierre, imprimiendo una tabla con origen, destino, distancia antes, distancia después, diferencia y estado.

VI-D. ¿Qué pruebas hicieron?

Se probó el cierre de Centro (que sí produce un impacto: algunas rutas se alargan, otras no cambian) verificando las diferencias exactas para los cinco pares; el cierre de Portal (un vértice periférico, donde ningún par de los probados cambia su distancia); y, con un grafo lineal $A-B-C-D-E$, que cerrar el vértice central C deja efectivamente desconectados los pares que dependían de él para comunicarse.

VI-E. ¿Qué limitaciones tiene la solución?

El impacto se mide solo sobre los pares de vértices que el usuario decide probar explícitamente, no sobre todos los pares posibles del grafo. Tampoco se contempla el cierre simultáneo de varios vértices o aristas, ni se recalculan rutas alternativas dinámicas como lo haría un sistema de transporte real.

VII. Coloreo de grafos

VII-A. ¿Qué problema resuelve el programa?

Asigna franjas horarias (colores) a un conjunto de cursos de forma que dos cursos con estudiantes en común nunca queden en la misma franja, evitando choques de horario en los exámenes.

VII-B. ¿Qué idea matemática usa?

Es un problema clásico de **coloreo de grafos**: cada curso es un vértice, y una arista entre dos cursos indica que comparten estudiantes (es decir, no pueden tener examen al mismo tiempo). Un coloreo válido asigna un color a cada vértice de modo que vértices adyacentes nunca compartan color. El programa usa un **algoritmo voraz** (*greedy*): recorre los vértices en un orden fijo y, para cada uno, le asigna el menor color que no esté siendo usado por ninguno de sus vecinos ya coloreados.

VII-C. ¿Cómo se ejecuta?

```
1 python3 src/grafos/coloreo_grafos.py
```

El menú colorea un grafo de ejemplo de 10 cursos (GRAFO_EJEMPLO), verifica que el coloreo sea válido y muestra cuántos colores se usaron y qué cursos quedaron en cada uno.

VII-D. ¿Qué pruebas hicieron?

Se verificó que el grafo de ejemplo tiene al menos 10 vértices, como exige el enunciado; que el coloreo producido es válido, es decir que ningún par de vértices adyacentes recibe el mismo color; y que todos los vértices del grafo reciben un color (ninguno queda sin asignar).

VII-E. ¿Qué limitaciones tiene la solución?

El algoritmo voraz siempre produce un coloreo válido, pero no garantiza usar el menor número posible de colores (el número cromático del grafo): el resultado depende del orden en que se recorren los vértices, y un orden distinto puede usar más o menos colores para el mismo grafo. Encontrar el número cromático óptimo es, en general, un problema NP-difícil.

VIII. Tablas de verdad y circuitos lógicos

VIII-A. ¿Qué problema resuelve el programa?

Genera automáticamente la tabla de verdad de una expresión booleana con hasta cuatro variables y conectivos

AND, OR, NOT y XOR, y permite evaluarla en una entrada concreta.

VIII-B. ¿Qué idea matemática usa?

Se apoya en **álgebra de Boole**: cada variable proposicional toma valores en $\{0, 1\}$ (falso/verdadero) y las expresiones se construyen combinando esas variables con los operadores lógicos booleanos. Para generar la tabla completa se enumeran, con `itertools.product`, todas las combinaciones posibles de valores de verdad de las variables usadas (2^n filas para n variables) y se evalúa la expresión en cada una. El programa incluye las tres expresiones pedidas: $(A \wedge B) \vee (\neg C)$, $(A \oplus B) \wedge C$ y $(A \vee B) \wedge (\neg A \vee C)$.

VIII-C. ¿Cómo se ejecuta?

```
python3 src/boole_shannon_compucuantic/tablas_verdad.py
```

Imprime la tabla de verdad completa de cada una de las tres expresiones, evalúa las tres en una entrada fija de ejemplo, y finalmente pide al usuario ingresar valores para A, B, C, D y muestra el resultado de cada expresión en esa entrada.

VIII-D. ¿Qué pruebas hicieron?

Para cada una de las tres expresiones se verificaron manualmente cuatro combinaciones de entrada representativas (casos verdaderos y falsos), comprobando que el resultado calculado coincide con la evaluación lógica esperada de la fórmula.

VIII-E. ¿Qué limitaciones tiene la solución?

Las expresiones están definidas como funciones de Python fijas dentro del código (no se parsea una fórmula escrita como texto libre), por lo que agregar una expresión nueva requiere escribir una función adicional en lugar de simplemente digitalarla. Además, el número de filas de la tabla crece exponencialmente con el número de variables (2^n), lo cual es inherente al problema y no una limitación evitable del programa.

Relación entre una tabla de verdad y un circuito lógico

Cada conectivo booleano (AND, OR, NOT, XOR) corresponde exactamente a una compuerta lógica en electrónica digital. Una expresión booleana con n variables se puede construir físicamente conectando esas compuertas, y la tabla de verdad de la expresión describe exactamente el comportamiento eléctrico del circuito resultante: cada fila indica qué salida (voltaje alto/bajo)

produce el circuito para cada combinación posible de entradas.

IX. Simplificación booleana

IX-A. ¿Qué problema resuelve el programa?

Dada una función booleana descrita por sus minterminos, encuentra una expresión equivalente en forma suma de productos que use la menor cantidad posible de literales y términos, lo que en un circuito real se traduce en menos compuertas.

IX-B. ¿Qué idea matemática usa?

Implementa una versión del método de **Quine–McCluskey**. Cada mintermino se representa como una cadena binaria de n bits. Dos términos que difieren en exactamente un bit se pueden *combinar*, reemplazando ese bit por un guion (-), ya que esa variable resulta irrelevante para que la expresión sea verdadera (esto es una aplicación directa de la ley de absorción / consenso del álgebra de Boole: $xy \vee x\bar{y} = x$). El proceso se repite agrupando términos por niveles hasta que ningún par se pueda combinar más; los términos que sobreviven sin combinarse son los **implicantes primos**. Luego se construye una tabla de cobertura y se seleccionan primero los implicantes *esenciales* (los únicos que cubren cierto mintermino) y después, de forma voraz, los que cubren más minterminos pendientes, hasta cubrir todos. Finalmente se verifica la equivalencia comparando la tabla de verdad de la expresión original contra la de la expresión simplificada.

IX-C. ¿Cómo se ejecuta?

```
1 python3 src/boole_shannon_compucuantic/
  simplificacion_booleana.py
```

Corre tres casos de ejemplo: el caso sugerido en el enunciado (3 variables, minterminos $\{1, 3, 5, 7\}$), otro con 3 variables y minterminos $\{0, 2, 4, 6\}$, y uno con 4 variables. Para cada caso imprime la expresión simplificada y una tabla que compara, fila por fila, la tabla de verdad original contra la de la expresión simplificada.

IX-D. ¿Qué pruebas hicieron?

Se probaron las funciones auxiliares por separado: que un número se convierte correctamente a binario con ceros a la izquierda; que dos términos que difieren en un solo bit sí se combinan (por ejemplo 101 y 001 dan -01); y que términos con más de una diferencia, o términos idénticos, no se combinan. El caso de prueba sugerido por el enunciado (minterminos $\{1, 3, 5, 7\}$ con variables A, B, C) produce una expresión equivalente a C , y el

propio programa verifica automáticamente que su tabla de verdad coincide con la de los minterminos originales.

IX-E. ¿Qué limitaciones tiene la solución?

La selección final de implicantes primos usa un criterio voraz para los minterminos no esenciales, lo que en algunos casos puede no producir la expresión con el mínimo absoluto de literales (el problema general de cobertura mínima de conjuntos es NP-difícil). Un **mintermino** es una combinación específica de valores de las variables para la cual la función vale 1; dos expresiones son equivalentes si, y solo si, producen el mismo valor de salida para absolutamente todas las combinaciones posibles de entrada, es decir, si tienen la misma tabla de verdad.

X. Shannon: medir información en un mensaje

X-A. ¿Qué problema resuelve el programa?

Cuantifica cuánta incertidumbre (información) contiene un mensaje de texto, y permite comparar qué tan "predecible" es un texto frente a otro.

X-B. ¿Qué idea matemática usa?

Usa la **entropía de Shannon**. Primero se calcula la frecuencia de cada símbolo del texto y, a partir de ella, su probabilidad empírica $p_i = \text{frecuencia}_i / \text{longitud del texto}$. La entropía se define como

$$H = - \sum_i p_i \log_2(p_i),$$

que mide, en bits, la incertidumbre promedio por símbolo. Un texto muy repetitivo concentra casi toda la probabilidad en pocos símbolos y tiene entropía baja (cercana a 0 si es un único símbolo repetido); un texto con símbolos muy variados y equiprobables tiene entropía alta.

X-C. ¿Cómo se ejecuta?

```
1 python3 src/boole_shannon_compucuantic/shannon.py
```

El programa pide dos textos por teclado, calcula la entropía de cada uno y muestra cuál de los dos tiene mayor entropía.

X-D. ¿Qué pruebas hicieron?

Se verificó que la función de frecuencias cuenta correctamente cada símbolo (incluyendo los espacios como símbolo válido), y que un texto formado por un único sím-

bolo repetido produce un diccionario de frecuencias con esa única entrada. Manualmente se comparó un mensaje muy repetitivo (por ejemplo "AAAAAAA") contra uno variado (por ejemplo un fragmento de texto normal), confirmando que el primero obtiene una entropía mucho menor que el segundo.

X-E. ¿Qué limitaciones tiene la solución?

La entropía se calcula a nivel de símbolo (carácter) de forma independiente, sin tener en cuenta la dependencia entre símbolos consecutivos (por ejemplo, que en español la letra "q" casi siempre va seguida de "u"); una entropía de orden superior, condicionada al contexto, daría una estimación más ajustada de la redundancia real del lenguaje. Además, la entropía mide incertidumbre estadística, no longitud del texto: dos textos de longitudes muy distintas pueden tener entropías por símbolo similares si usan sus símbolos con proporciones parecidas.

XI. Primer simulador cuántico: bits, qubits y mediciones

XI-A. ¿Qué problema resuelve el programa?

Simula, con vectores y matrices simples, cómo se comporta un único qubit al aplicarle compuertas cuánticas básicas y al medirlo, sin necesidad de acceso a un computador cuántico real.

XI-B. ¿Qué idea matemática usa?

Un qubit se representa como un vector de estado $\begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ que combina las bases $|0\rangle$ y $|1\rangle$, donde $|\alpha|^2$ y $|\beta|^2$ son las probabilidades de medir 0 o 1 respectivamente (y deben sumar 1). Aplicar una compuerta cuántica corresponde a multiplicar ese vector por una matriz 2×2 : la compuerta X (negación cuántica) intercambia las amplitudes, Z deja $|0\rangle$ igual y cambia el signo de $|1\rangle$, y H (Hadamard) pone el qubit en superposición usando el factor $1/\sqrt{2}$:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Una vez calculado el estado, las probabilidades se obtienen elevando al cuadrado cada amplitud, y una medición se simula generando un número aleatorio uniforme en $[0, 1)$ y comparándolo contra esa probabilidad, repitiendo el proceso 1000 veces para observar las frecuencias.

XI-C. ¿Cómo se ejecuta?

```
python3 src/boole_shannon_compucuantic/
simulador_cuantico.py
```

Corre los tres casos obligatorios del enunciado en orden: $X|0\rangle$, $H|0\rangle$ (con sus probabilidades y 1000 mediciones simuladas) y $HH|0\rangle$.

XI-D. ¿Qué pruebas hicieron?

Se comprobó que $X|0\rangle = |1\rangle$ y $X|1\rangle = |0\rangle$; que Z deja $|0\rangle$ sin cambios y le invierte el signo a $|1\rangle$; que $H|0\rangle$ produce probabilidades cercanas a 50% y 50% (verificado con precisión numérica de 9 decimales, ya que $1/\sqrt{2}$ al cuadrado es exactamente 0.5); que aplicar H dos veces seguidas sobre $|0\rangle$ recupera el estado original $|0\rangle$ (salvo errores de redondeo de punto flotante); y que pedir una compuerta desconocida lanza un error controlado.

XI-E. ¿Qué limitaciones tiene la solución?

Solo simula un único qubit (no hay múltiples qubits ni entrelazamiento), y las amplitudes se modelan como números reales, no complejos, por lo que no se pueden representar todas las compuertas cuánticas posibles (por ejemplo, la compuerta de fase S requiere amplitudes complejas). La diferencia clave con un computador cuántico real es que aquí las probabilidades se calculan de forma exacta con álgebra lineal clásica y las "mediciones" son solo un muestreo pseudoaleatorio sobre esas probabilidades; en un computador cuántico real, la medición colapsa físicamente el estado del qubit y está sujeta a ruido y decoherencia del hardware, efectos que esta simulación no reproduce.

XII. Conclusiones

Trabajar los diez puntos de este taller mostró que temas que en el curso se ven por separado –congruencias, grafos, álgebra booleana, teoría de la información y álgebra lineal– comparten una misma base: representar un problema con estructuras discretas y resolverlo con un procedimiento preciso y verificable. Implementar cada solución obligó a entender no solo la fórmula matemática detrás, sino también sus condiciones de validez (por qué RSA necesita que e y $\varphi(n)$ sean primos relativos, por qué Dijkstra requiere pesos no negativos, por qué el coloreo voraz no siempre es óptimo), lo cual reforzó la comprensión teórica del curso mucho más que memorizar las definiciones. Las pruebas automatizadas, además, fueron clave para confirmar que cada implementación cumplía exactamente lo que el enunciado esperaba, incluyendo los casos límite.

XIII. Uso de herramientas

Para la elaboración de este documento de explicación y del `README.md` del repositorio se usó como apoyo un asistente de inteligencia artificial (Claude, de Anthropic), a partir del código ya escrito por los integrantes del grupo, con el fin de organizar y redactar la documentación de forma más clara y consistente con el enunciado del taller. El diseño, la implementación y las pruebas del código fueron desarrollados y son comprendidos por los integrantes, quienes pueden explicar cualquier función, prueba o decisión de los algoritmos presentados.